

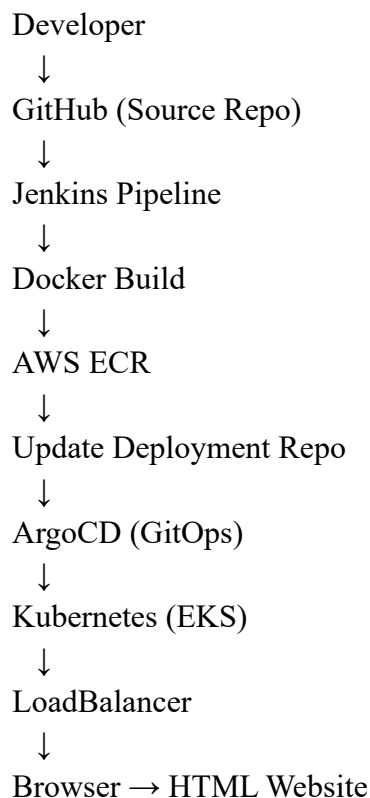
END-TO-END CI/CD PIPELINE IMPLEMENTATION USING GITHUB, JENKINS, SONARQUBE, TRIVY, DOCKER, AWS ECR, ARGO CD, KUBERNETES, AND AMAZON EKS

Introduction

This project demonstrates the implementation of an end-to-end CI/CD pipeline using Jenkins, Docker, SonarQube, Trivy, AWS ECR, Argo CD, and Amazon EKS. The goal of the project is to automate the software delivery process, ensuring faster, secure, and reliable application deployments.

The Continuous Integration (CI) phase uses Jenkins to automate source code checkout, code quality analysis, Docker image creation, vulnerability scanning, and image storage in AWS ECR. The Continuous Deployment (CD) phase follows the GitOps approach, where Argo CD continuously monitors Kubernetes manifests stored in a Git repository and automatically deploys changes to an Amazon EKS cluster.

FINAL ARCHITECTURE



HANDS-ON FOR END-TO-END GITOPS-BASED CI/CD PIPELINE ON AWS EKS

CONTINUOUS INTEGRATION PART

Screenshot: Launch Project Server Instance

- Launch an EC2 instance.
- Enter the instance name as **Project-Server**.
- Select the **Ubuntu Server AMI**.

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Recents

My AMIs

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 20.04 LTS (ARM) SSD Volume Type

Free tier eligible

Screenshot: Configure Instance Type and Key Pair

- Select the required **Instance Type**.
- Select the **Key Pair**.
- Continue with the instance configuration.

▼ Instance type [Info](#) | [Get advice](#)

Instance type

m7i-flex.large

Free tier eligible

Family: m7i-flex 2 vCPU 8 GiB Memory Current generation: true

On-Demand SUSE base pricing: 0.15705 USD per Hour On-Demand RHEL base pricing: 0.12955 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.10425 USD per Hour On-Demand Linux base pricing: 0.10075 USD per Hour

On-Demand Windows base pricing: 0.18815 USD per Hour

☐ All generations

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

[Create new key pair](#)

Screenshot: Configure Security Group

- Select the Security Group.
- Allow **All TCP** traffic.
- Save the configuration and continue.

Firewall (security groups) [Info](#)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group ☒ Select existing security group

Common security groups [Info](#)

Select security groups

[Compare security group rules](#)

Security groups that you add or remove here will be added to or removed from all your network interfaces.

▼ Configure storage [Info](#) Advanced

1x GiB Root volume, 3000 IOPS, Not encrypted

Screenshot: Launch EC2 Instance

- Review the instance configuration.
- Click **Launch Instance**.
- EC2 instance created successfully.

Instances (1/1) [Info](#) Last updated less than a minute ago [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Saved filter sets [Choose filter set](#)

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
☒	project-server	i-02d2e529009aa0b01	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-13-233-144-1.ap-s...	13.233.144.1	-

Screenshot: Install Java

- Connect to the **Project-Server** instance.
- Install **Java**.
- Verify the installation.

```
ubuntu@ip-172-31-11-158:~$  
ubuntu@ip-172-31-11-158:~$ sudo apt update  
sudo apt install fontconfig openjdk-21-jre  
java -version  
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu InRelease  
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu InRelease [137 kB]  
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu InRelease [136 kB]  
Get:4 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu InRelease [16.3 MB]  
Get:5 http://security.ubuntu.com/ubuntu InRelease [137 kB]
```

Screenshot: Install and Start Jenkins

- Install **Jenkins**.
- Start the Jenkins service.
- Check the Jenkins service status.

```
ubuntu@ip-172-31-11-158:~$
ubuntu@ip-172-31-11-158:~$ sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins
--2026-06-16 05:02:59-- https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 151.101.210.133, 2a04:4e42:31::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[151.101.210.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1680 (1.6K) [application/octet-stream]
Saving to: '/etc/apt/keyrings/jenkins-keyring.asc'
```

```
root@ip-172-31-11-158:/home/ubuntu# systemctl start jenkins
root@ip-172-31-11-158:/home/ubuntu# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-16 05:03:32 UTC; 3min 19s ago
  Invocation: dbef42e856ed4f7f8e95e377c53b16bd
   Main PID: 4254 (java)
```

Screenshot: Install Docker and Configure Jenkins

- Update the package repository.
- Install **Docker**.
- Add the **Jenkins** user to the Docker group.
- Restart the Jenkins service.
- Docker configured successfully for Jenkins.

Screenshot: Run SonarQube Container

- Run the SonarQube Docker container.
- Expose SonarQube on port **9000**.
- SonarQube container started successfully.

```
root@ip-172-31-11-158:/home/ubuntu#
root@ip-172-31-11-158:/home/ubuntu# docker run -d --name sonarqube -p 9000:9000 sonarqube:latest
Unable to find image 'sonarqube:latest' locally
latest: Pulling from library/sonarqube
d264b91a55d8: Pull complete
cb259a83ac3d: Pull complete
72c6a52a9f2d: Pull complete
d385704da112: Downloading [=====>] 937.1MB/937.1MB
```

Screenshot: Install Sonar Scanner

- Download Sonar Scanner.
- Install and extract Sonar Scanner.
- Configure the Sonar Scanner path.
- Sonar Scanner installed successfully.

```
root@ip-172-31-11-158:/home/ubuntu#
root@ip-172-31-11-158:/home/ubuntu# wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-5.0.1.3006-linux.zip
sudo apt install -y unzip
unzip sonar-scanner-cli-5.0.1.3006-linux.zip
sudo mv sonar-scanner-5.0.1.3006-linux /opt/sonar-scanner

export PATH=$PATH:/opt/sonar-scanner/bin
--2026-06-16 05:12:09-- https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-5.0.1.3006-linux.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 13.225.5.16, 13.225.5.20, 13.225.5.56, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)[13.225.5.16]:443... connected.
HTTP request sent, awaiting response... 200 OK
```

Screenshot: Install Trivy

- Add the Trivy repository.
- Install Trivy.
- Verify the installation.
- Trivy installed successfully.

```
root@ip-172-31-11-158:/home/ubuntu#
root@ip-172-31-11-158:/home/ubuntu# sudo apt-get update
sudo apt-get install -y wget gnupg lsb-release

# Create keyrings directory
sudo mkdir -p /etc/apt/keyrings

# Download and add Trivy GPG key
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | \
gpg --dearmor | sudo tee /etc/apt/keyrings/trivy.gpg > /dev/null

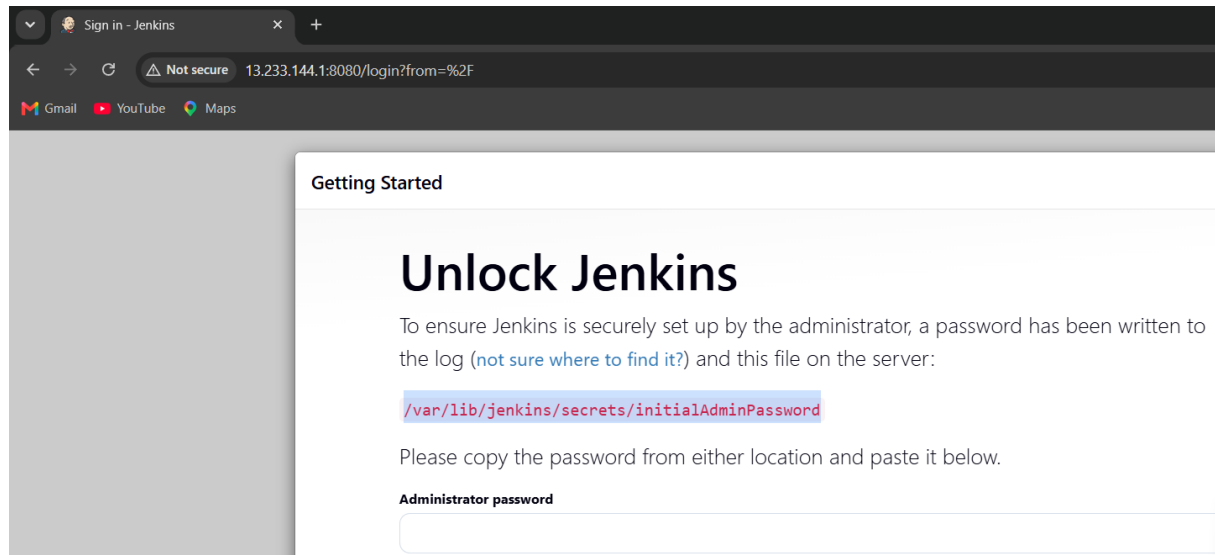
# Add Trivy repository
echo "deb [signed-by=/etc/apt/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" | \
sudo tee /etc/apt/sources.list.d/trivy.list

# Update packages
sudo apt-get update

# Install Trivy
sudo apt-get install -y trivy
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
```

Screenshot: Access Jenkins

- Copy the **Public IP** of the server.
- Open a browser and navigate to **PublicIP:8080**.
- Jenkins setup page displayed successfully.



Screenshot: Get Jenkins Initial Password

- Copy the Jenkins password file path from the setup page.
- Run the command to view the password.
- Copy the Jenkins initial admin password successfully.

```
root@ip-172-31-11-158:/home/ubuntu#
root@ip-172-31-11-158:/home/ubuntu# cat /var/lib/jenkins/secrets/initialAdminPassword
e02bb4ed321d42dfb79af9cae4730483
```

Screenshot: Unlock Jenkins

- Paste the initial admin password.
- Click **Continue**.
- Jenkins unlocked successfully.

`/var/lib/jenkins/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Screenshot: Create Admin User

- Enter the username and password.
- Enter the full name and email address.
- Click **Save and Continue**.
- Admin user created successfully.

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

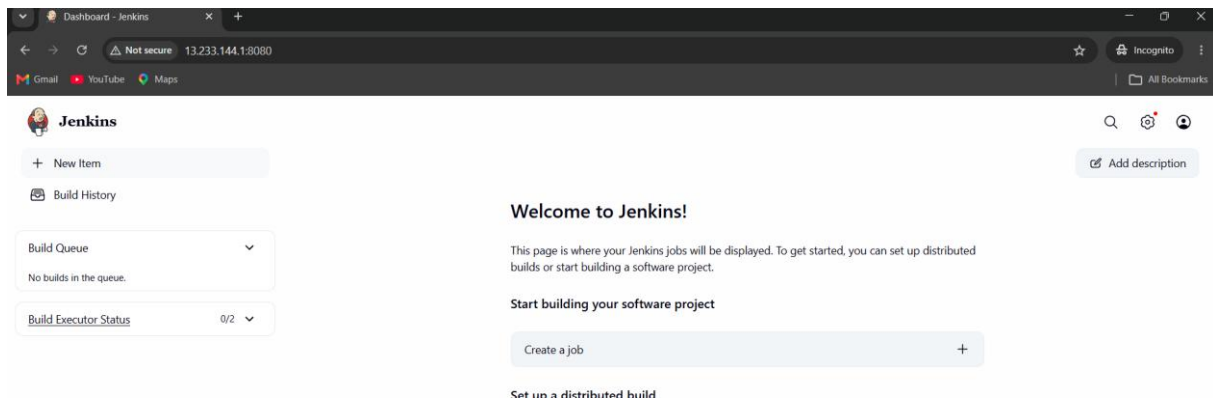
E-mail address

Jenkins 2.555.3

[Skip and continue as admin](#) [Save and Continue](#)

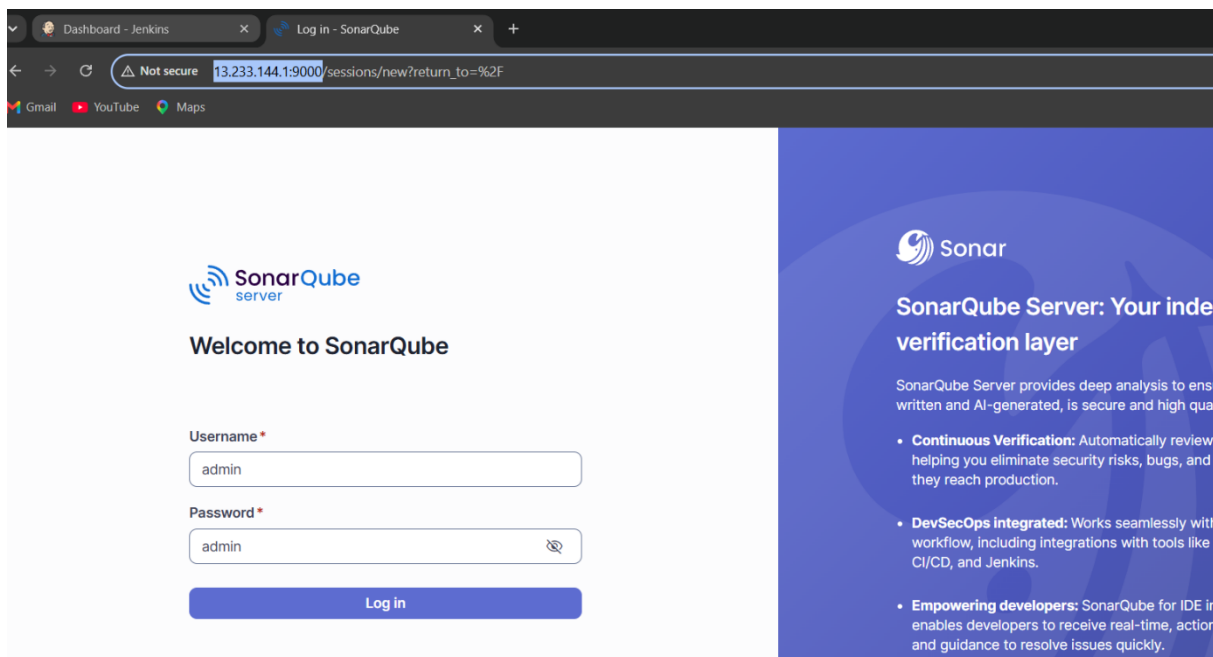
Screenshot: Jenkins Dashboard

- Jenkins setup completed successfully.
- Jenkins Home Page displayed successfully.



Screenshot: Access SonarQube

- Copy the **Public IP** of the server.
- Open a browser and navigate to **PublicIP:9000**.
- Login using **admin / admin**.
- SonarQube login successful.



Screenshot: Change SonarQube Password

- Enter a new password.
- Confirm the new password.
- Click **Update**.
- Password updated successfully.

Update your password

 This account should not use the default password.

Enter a new password

Old Password *

.....

Password *

.....

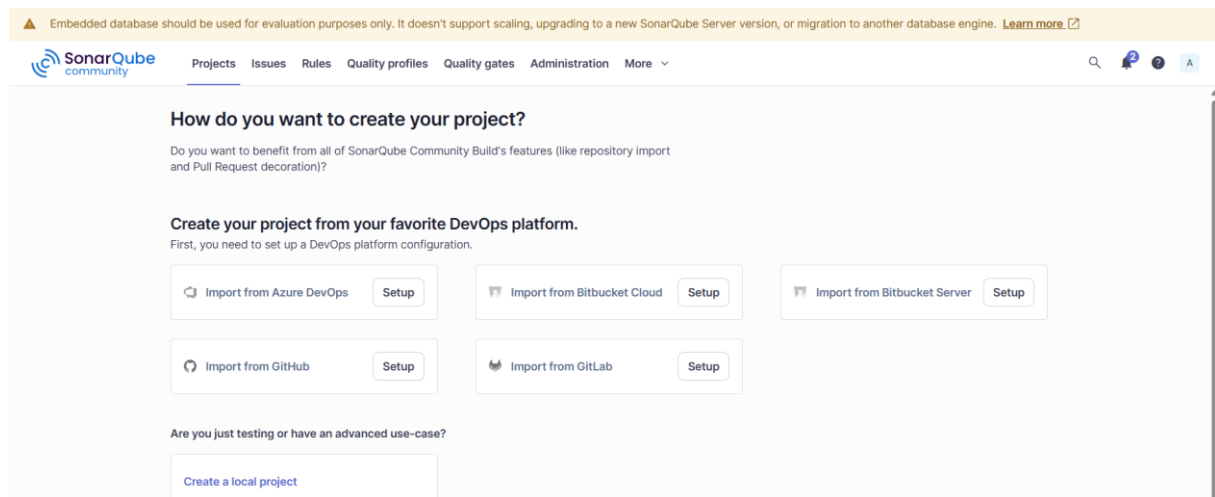
Confirm Password *

.....

Update

Screenshot: SonarQube Dashboard

- SonarQube setup completed successfully.
- SonarQube Home Page displayed successfully.



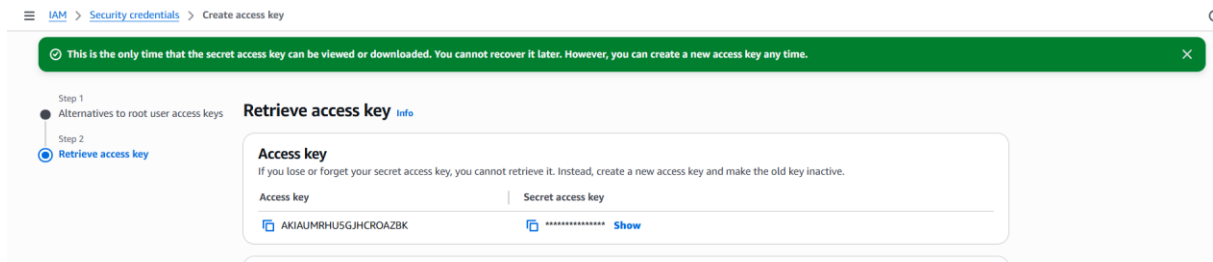
Screenshot: Install kubectl

- Install **kubectl** on the Project Server.
- Verify the installation.

```
root@ip-172-31-11-158:/home/ubuntu#
root@ip-172-31-11-158:/home/ubuntu# curl -o kubectl https://s3.us-west-2.amazonaws.com/amazon-eks/1.29.0/2024-01-04/bin/linux/amd64/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin
kubectl version --client
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed
100 47.40M 100 47.40M    0     0  9.25M    0  0:05  0:05  0:00  7.77M
Client Version: v1.29.0-eks-5e0fdde
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
```

Screenshot: Create AWS Access Key

- Open the AWS Console.
- Create an **Access Key** for the IAM user.
- Copy the Access Key and Secret Access Key.
- Access keys created successfully.



Screenshot: Install AWS CLI

- Download and install **AWS CLI**.
- Run AWS configuration.
- Paste the Access Key and Secret Access Key.
- AWS CLI configured successfully.

```
root@ip-172-31-11-158:/home/ubuntu# aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAUMRHUSGJHCROAZBK
AWS Secret Access Key [None]: 1DnJnvi/YEuE7+HIF/VGM1NMzE9rII4Jf012rbHY
Default region name [None]:
Default output format [None]:
root@ip-172-31-11-158:/home/ubuntu#
```

Screenshot: Install Helm

- Install **Helm**.
- Verify the installation.

```
root@ip-172-31-11-158:/home/ubuntu# curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total     Spent    Left     Speed
100 11929 100 11929    0     0  1.05M      0      0      0
Downloading https://get.helm.sh/helm-v3.21.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
version.BuildInfo{Version:"v3.21.1", GitCommit:"c56dd0095fd76da5d7b30ecdf506103e7f26745e", GitTreeState:"clean", GoVersion:"go1.26.4"}
root@ip-172-31-11-158:/home/ubuntu#
```

CODE CHECKOUT STAGE

Screenshot: Create Jenkins Pipeline Project

- Open the Jenkins Dashboard.
- Click **New Item**.
- Enter the project name as **my-project**.
- Select **Pipeline**.
- Click **OK**.



Jenkins

+ New Item

Build History

New Item

Enter an item name

my-project

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Screenshot: Configure Code Checkout Stage

- Open **my-project** in Jenkins.
- Click **Configure**.
- Navigate to the **Pipeline** section.
- Paste the Pipeline script for code checkout.
- Click **Apply**.
- Click **Save**.
- Pipeline configuration updated successfully.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Checkout') {
6       steps {
7         git branch: 'main',
8             credentialsId: 'git-cred',
9             url: 'https://github.com/jaisurya003/application-code.git'
10        }
11      }
12    }
13  }
```

Save

Apply

Screenshot: Verify Project Creation

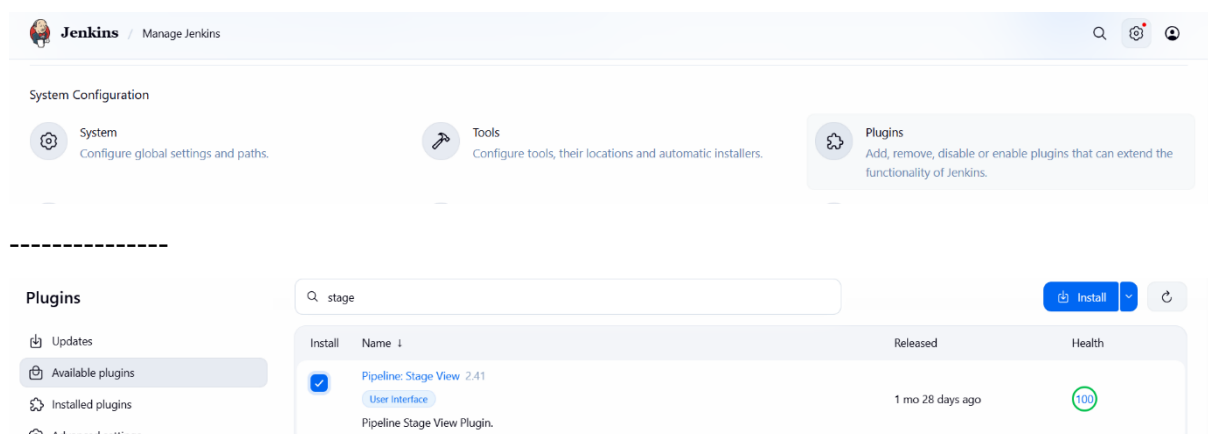
- Return to the Jenkins Dashboard.
- Verify that **my-project** is listed.



SETTING THE IMAGE FOR BUILD

Screenshot: Install Stage View Plugin

- Open **Manage Jenkins**.
- Click **Plugins**.
- Search for **Stage View Plugin**.
- Install the plugin successfully.



Screenshot: Configure Image Tag Build Stage

- Open **my-project**.
- Click **Configure**.
- Update the Pipeline script to include the **Set Image Tag** stage.
- Click **Apply**.
- Click **Save**.

Definition

Pipeline script

Script ?

```
16     }
17
18     stage('Set Image Tag') {
19         steps {
20             script {
21                 def shortHash = sh(
22                     script: "git rev-parse --short HEAD",
23                     returnStdout: true
24                 ).trim()
25
26                 env.IMAGE_TAG = "${BUILD_NUMBER}-${shortHash}"
27                 echo "Using image tag: ${env.IMAGE_TAG}"
28             }
29         }
30     }
```

Save

Apply

Screenshot: Verify Pipeline Execution

- Click **Build Now**.
- Open **Stage View**.
- Verify that **Checkout** and **Set Image Tag** stages completed successfully.
- Pipeline executed successfully.

 **Jenkins** / my-project

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

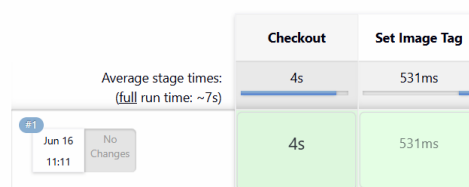
Rename

Pipeline Syntax

GitHub Hook Log

my-project

Stage View



Permalinks

Builds >

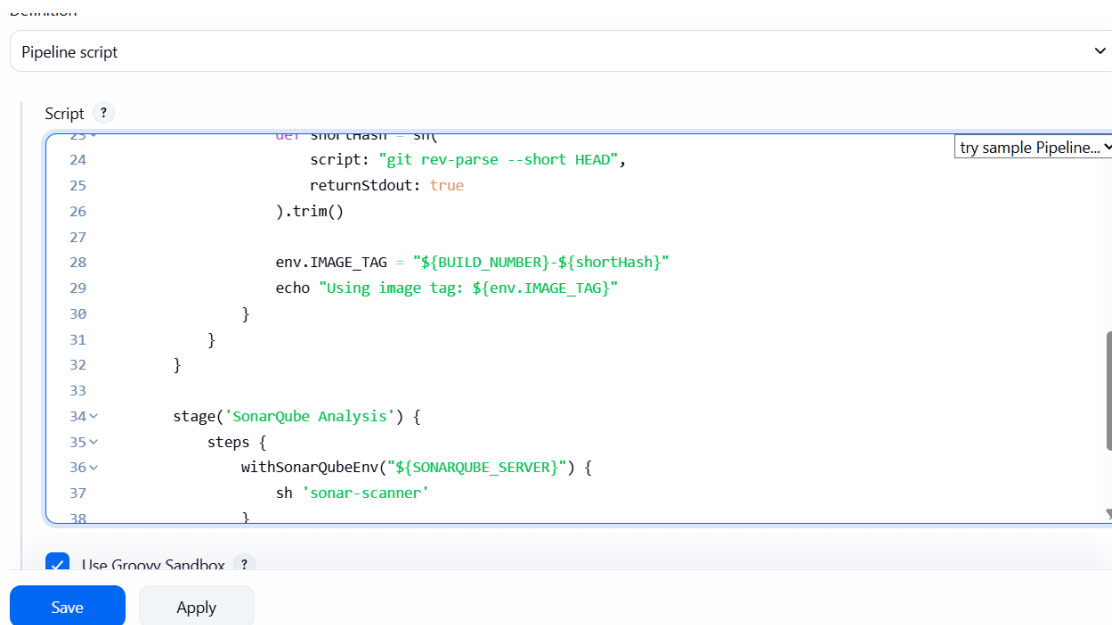
Today

#1 5:41 AM

ADDED THE SONARQUBE STAGE FOR SECURING THE APPLICATION

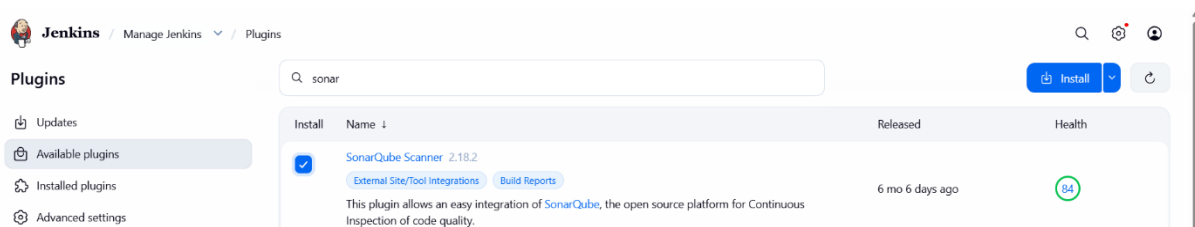
Screenshot: Add SonarQube Analysis Stage

- Open **my-project**.
- Click **Configure**.
- Update the Pipeline script to include the **SonarQube Analysis** stage.
- Click **Apply**.
- Click **Save**.



Screenshot: Install SonarQube Plugin

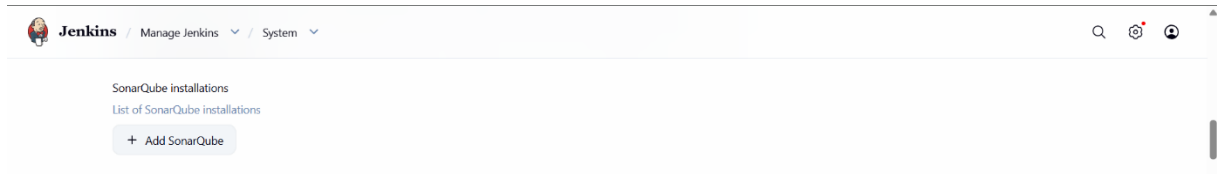
- Open **Manage Jenkins**.
- Click **Plugins**.
- Search for **SonarQube Scanner**.
- Install the plugin successfully.



Screenshot: Add SonarQube Server

- Open **Manage Jenkins**.
- Click **System**.

- Navigate to **SonarQube Servers**.
- Click **Add SonarQube**.
- SonarQube server configuration page opened successfully.



Screenshot: Configure SonarQube Server

- Enter the name as **SonarQubeServer**.
- Enter the SonarQube server URL.
- In **Authentication Token**, click **Add**.
- Select **Jenkins** scope and continue.

SonarQube installations
List of SonarQube installations

Name

Server URL
Default is http://localhost:9000

Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.

+ Add

Advanced

System
Global
Credentials that should be available everywhere.

Screenshot: Add SonarQube Token Credential

- Select **Secret Text** as the credential type.
- Proceed to create the credential.

Add Credentials

×

Select a type of credential



Username with password

Commonly used for authentication to services like Git, APIs, or registries.



GitHub App



SSH Username with private key



Secret file



Secret text



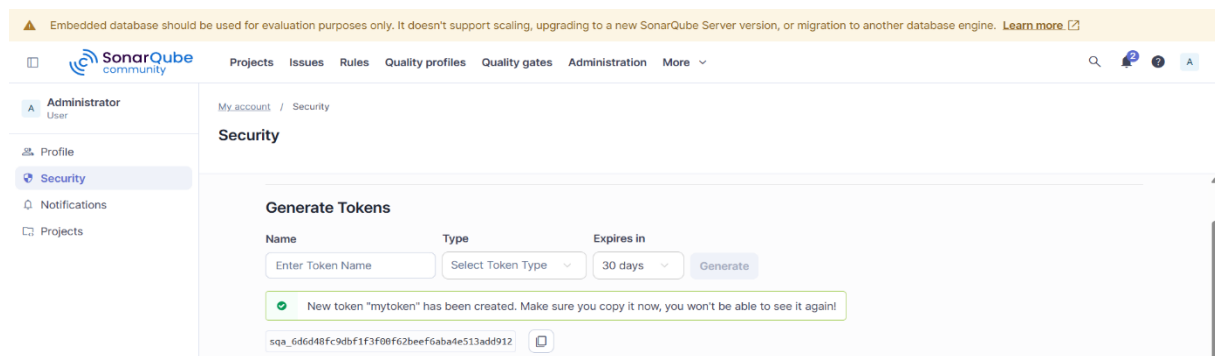
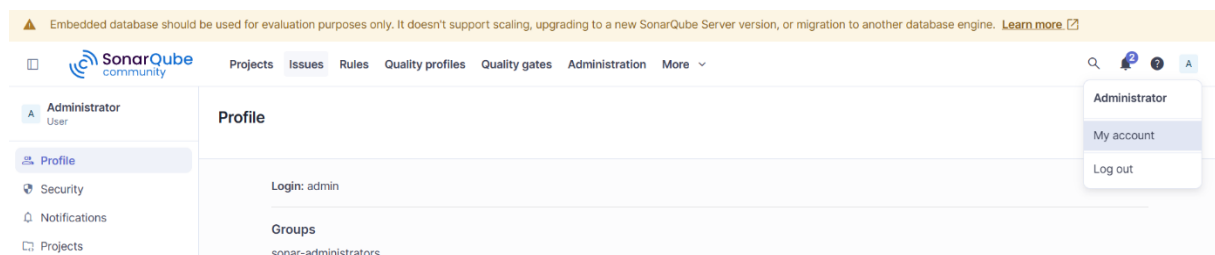
Certificate

Upload a PKCS#12 or PEM encoded certificate and private key.

Next

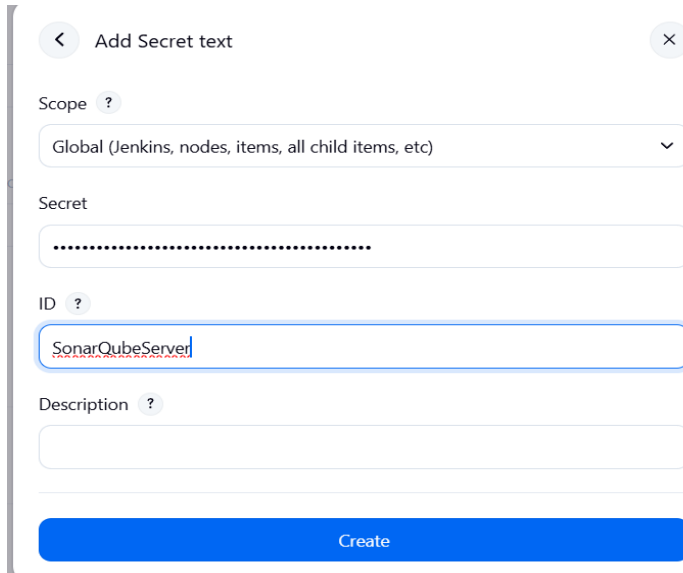
Screenshot: Generate SonarQube Token

- Open the SonarQube Dashboard.
- Click the **Profile** icon.
- Select **My Account**.
- Open the **Security** tab.
- Enter a token name.
- Click **Generate**.
- Copy the generated token successfully.



Screenshot: Create Jenkins Credential

- Paste the token into the **Secret** field.
- Enter the ID as **SonarQubeServer**.
- Click **Create**.
- SonarQube token credential created successfully.



< Add Secret text ×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Secret
.....

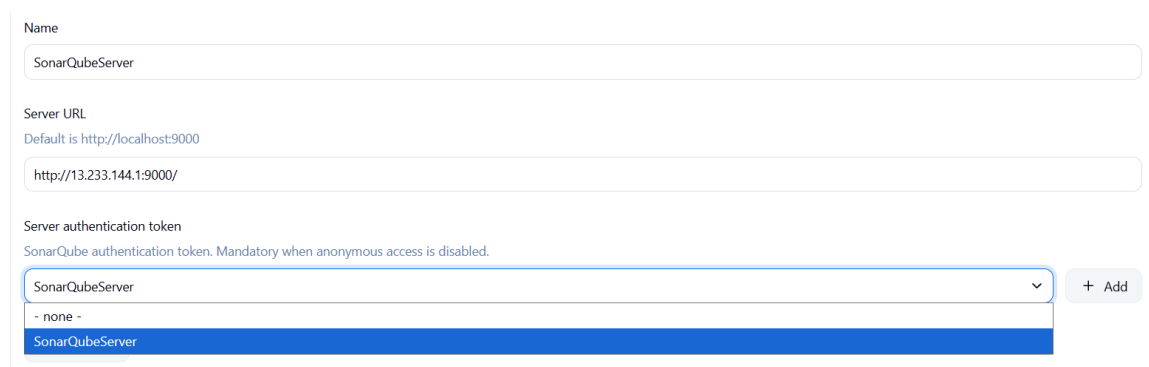
ID ?
SonarQubeServer

Description ?

Create

Screenshot: Select SonarQube Credential

- Return to the SonarQube server configuration.
- Select **SonarQubeServer** in the **Authentication Token** field.
- Save the configuration successfully.



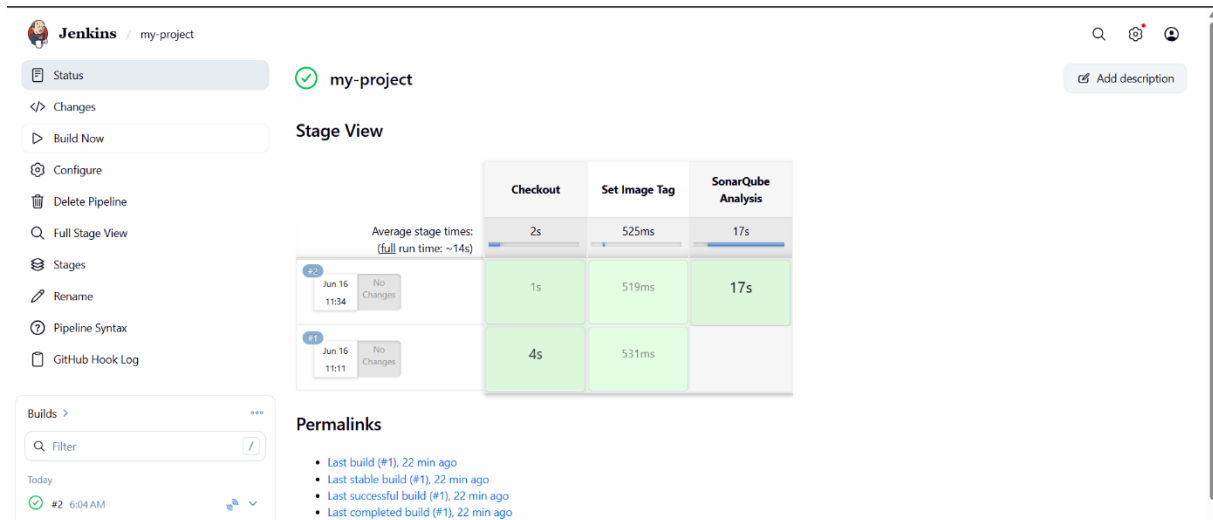
Name
SonarQubeServer

Server URL
Default is http://localhost:9000
http://13.233.144.1:9000/

Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.
SonarQubeServer ▼ + Add
- none -
SonarQubeServer

Screenshot: Run Jenkins Pipeline

- Open **my-project**.
- Click **Build Now**.
- Verify that **Checkout**, **Set Image Tag**, and **SonarQube Analysis** stages completed successfully.
- Pipeline executed successfully.



Screenshot: Verify SonarQube Project

- Open the **SonarQube Dashboard**.
- Navigate to **Projects**.
- Verify that the application project has been added.
- SonarQube analysis completed successfully and the project is displayed.



BUILD THE DOCKER IMAGE AND TRIVY IMAGE

Screenshot: Add Docker Build and Trivy Scan Stages

- Open **my-project**.
- Click **Configure**.
- Update the Pipeline script to include **Build Docker Image** and **Image Scan** stages.
- Click **Apply**.
- Click **Save**.

```
41
42 stage('Build Docker Image') {
43     steps {
44         echo "Using updated Image tag ${IMAGE_NAME}:${IMAGE_TAG}"
45
46         sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
47         sh "docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest"
48     }
49 }
50
51 stage('Image Scan') {
52     steps {
53         sh "trivy image ${IMAGE_NAME}:${IMAGE_TAG} > report.txt"
54     }
55 }
```

☒ Use Groovy Sandbox ?

Screenshot: Verify Docker Build and Image Scan

- Open **my-project**.
- Click **Build Now**.
- Verify that **Checkout**, **Set Image Tag**, **SonarQube Analysis**, **Build Docker Image**, and **Image Scan** stages completed successfully.
- Pipeline executed successfully.

The screenshot shows the Jenkins interface for a pipeline named 'my-project'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, SonarQube, Stages, Rename, Pipeline Syntax, and GitHub Hook Log. The main area displays the 'Stage View' of the pipeline execution. A table shows the duration of each stage for three builds. Below the table, the 'SonarQube Quality Gate' section indicates that the 'FastAPI Project' has passed.

	Checkout	Set Image Tag	SonarQube Analysis	Build Docker Image	Image Scan
Average stage times: (full run time: ~33s)	2s	499ms	15s	30s	25s
#3 Jun 16 11:38 No Changes	837ms	449ms	12s	30s	25s
#2 Jun 16 11:34 No Changes	1s	519ms	17s		
#1 Jun 16 11:11 No Changes	4s	531ms			

SonarQube Quality Gate

FastAPI Project **Passed**

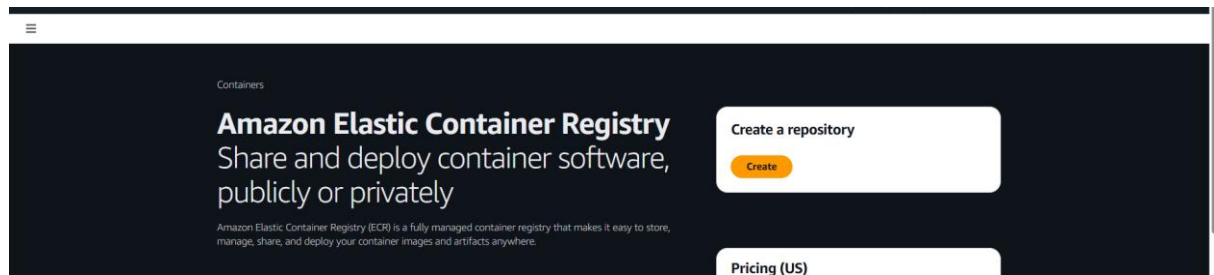
server-side processing: **Success**

PUSH TO ECR IN AWS

Screenshot: Create Amazon ECR Repository

- Open the AWS Console.
- Navigate to **Amazon ECR**.
- Click **Create Repository**.

- Enter the repository name as **my-ecr-repo**.
- Click **Create Repository**.
- ECR repository created successfully.



Amazon ECR > Private registry > Repositories > Create private repository

Create private repository

General settings

Repository name
Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

301804611986.dkr.ecr.ap-south-1.amazonaws.com/**my-ecr-repo**

11 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers, and special characters _-./.

Image tag settings [Info](#)

Image tag mutability
Choose the tag mutability setting.

☒ **Mutable**
Image tags can be overwritten.

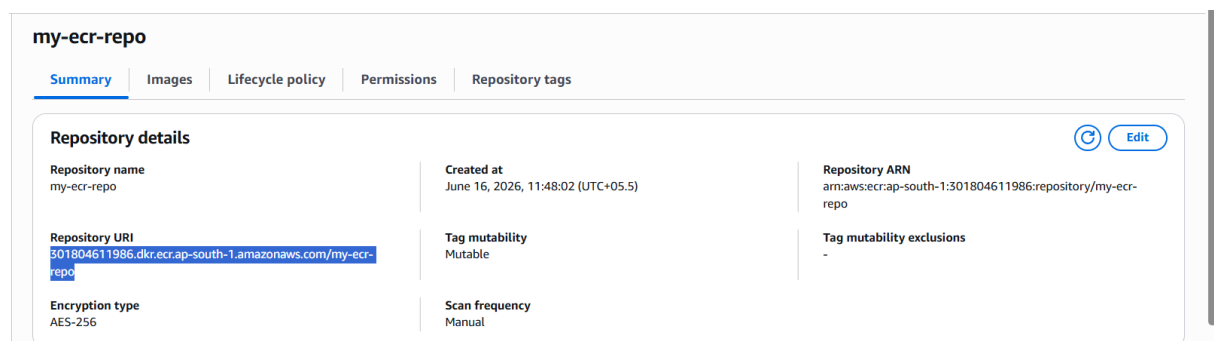
☐ **Immutable**
Image tags can't be overwritten.

Private repositories (1)

Repository name	URI	Created at	Tag immutability	Encryption type
my-ecr-repo	301804611986.dkr.ecr.ap-south-1.amazonaws.com/my-ecr-repo	June 16, 2026, 11:48:02 (UTC+05.5)	Mutable	AES-256

Screenshot: Copy ECR Repository URL

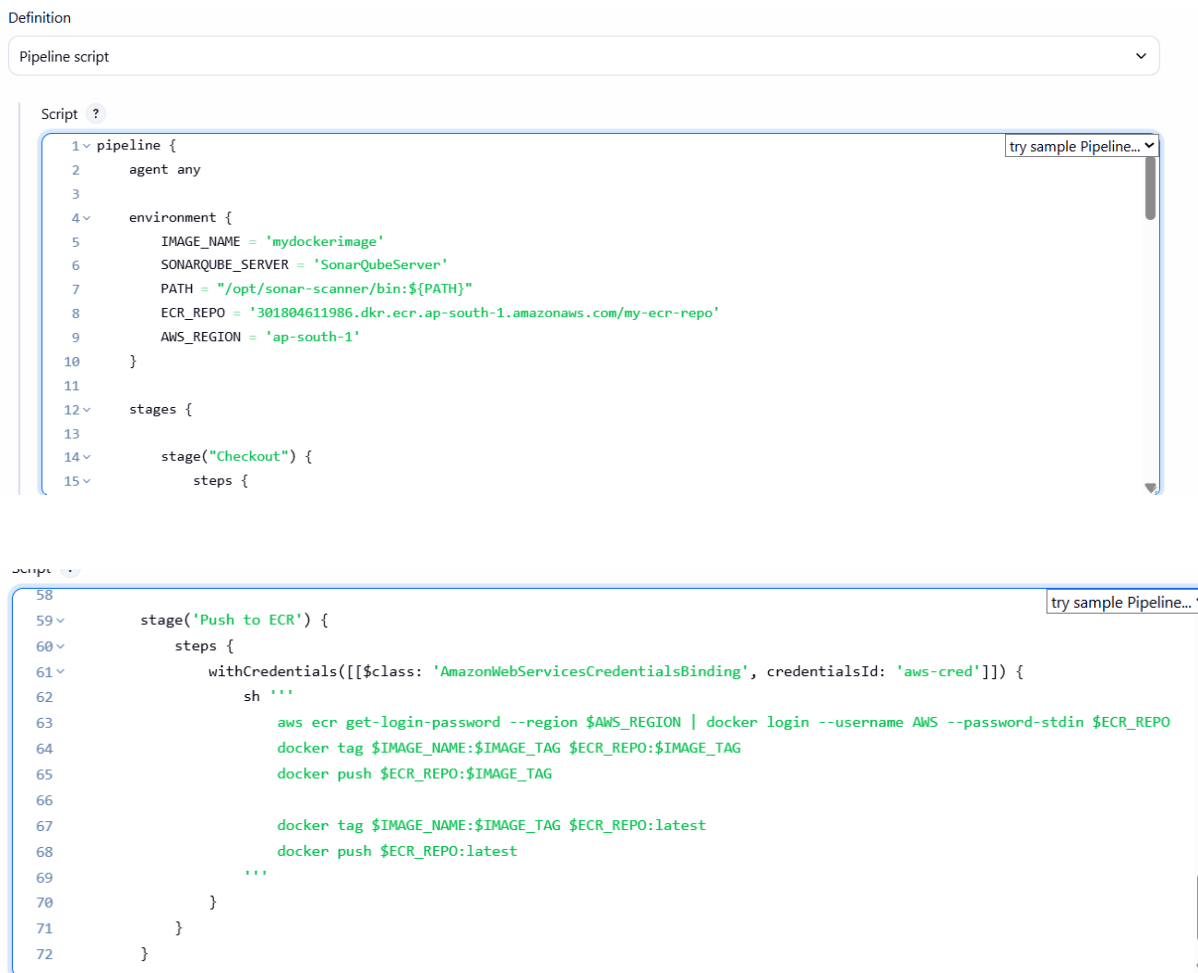
- Open the **my-ecr-repo** repository.
- Copy the Repository URI.



Screenshot: Add Push to ECR Stage

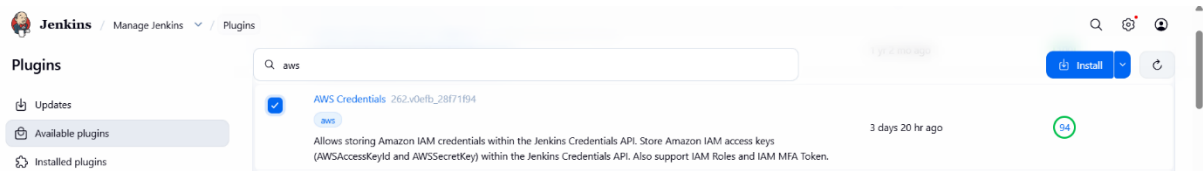
- Open **my-project**.

- Click **Configure**.
- Update the Pipeline script to include the **Push to ECR** stage.
- Replace the **ECR_REPO** value with the copied ECR Repository URL.
- Click **Apply**.
- Click **Save**.
- Pipeline configuration updated successfully.



Screenshot: Install AWS Credentials Plugin

- Open **Manage Jenkins**.
- Click **Plugins**.
- Search for **AWS Credentials Plugin**.
- Install the plugin successfully.



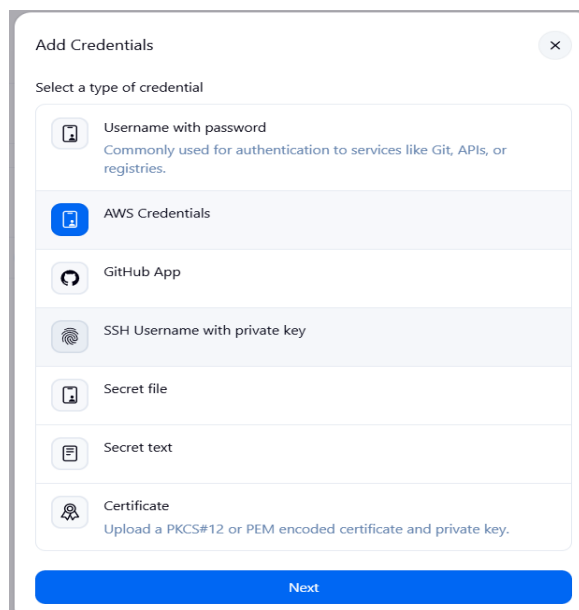
Screenshot: Add AWS Credentials

- Open **Manage Jenkins**.
- Click **Credentials**.
- Click **Add Credentials**.



Screenshot: Create AWS Credential

- Select **AWS Credentials**.
- next
- Enter the ID as **aws-cred**.
- Paste the AWS Access Key.
- Paste the AWS Secret Access Key.
- Click **Create**.
- AWS credential created successfully.



< Add AWS Credentials ×

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

ID ?
aws-cred

Description ?

Access Key ID ?
AKIAUMRHU5GJHCROAZBK

Secret Access Key
.....

! Please specify the Secret Access Key

IAM Role Support
Advanced ▼

Create

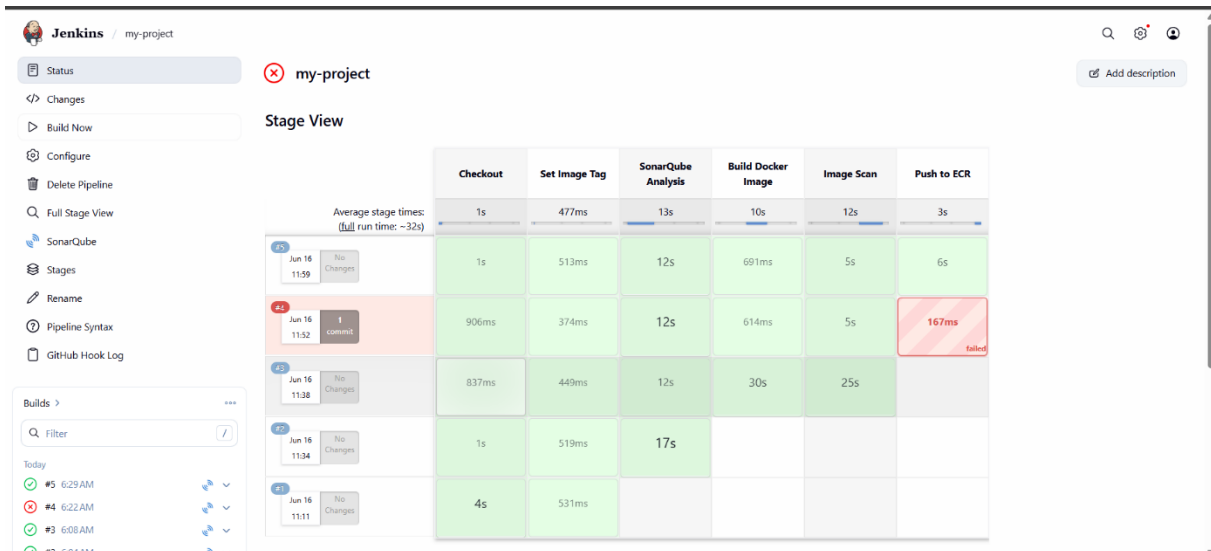
Screenshot: Verify AWS Credential

- Open **Manage Jenkins** → **Credentials**.
- Verify that **aws-cred** is available.

Credentials		+ Add Credentials	?
	SonarQubeServer System - Global		
	aws-cred AKIAUMRHU5GJHCROAZBK System - Global		

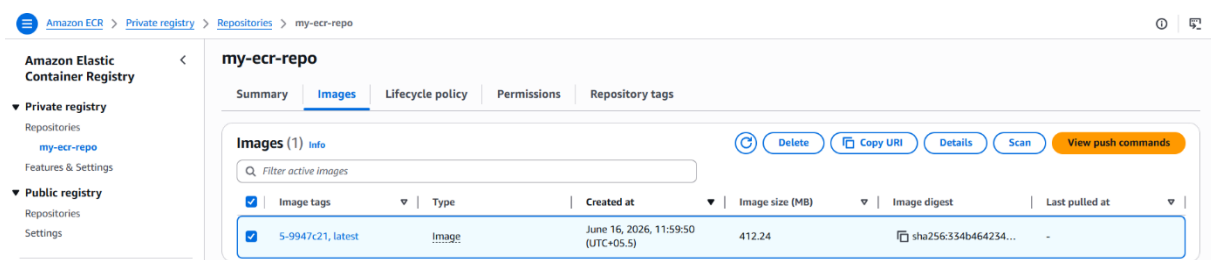
Screenshot: Verify Push to ECR

- Open **my-project**.
- Click **Build Now**.
- Verify that all stages including **Push to ECR** completed successfully.
- Pipeline executed successfully.



Screenshot: Verify Image in ECR

- Open Amazon ECR.
- Select **my-ecr-repo**.
- Navigate to **Images**.
- Verify that the Docker image is available.
- Image pushed to ECR successfully.
-



```
root@ip-172-31-11-158:/home/ubuntu# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
301804611986.dkr.ecr.ap-south-1.amazonaws.com/my-ecr-repo:5-9947c21	334b46423466	1.61GB	412MB	
301804611986.dkr.ecr.ap-south-1.amazonaws.com/my-ecr-repo:latest	334b46423466	1.61GB	412MB	
mydockerimage:3-96e184c	334b46423466	1.61GB	412MB	
mydockerimage:4-9947c21	334b46423466	1.61GB	412MB	
mydockerimage:5-9947c21	334b46423466	1.61GB	412MB	
mydockerimage:latest	334b46423466	1.61GB	412MB	
python:3.9	da5aee29682d	1.6GB	423MB	
sonarqube:latest	4671314254b0	2.53GB	1.08GB	U

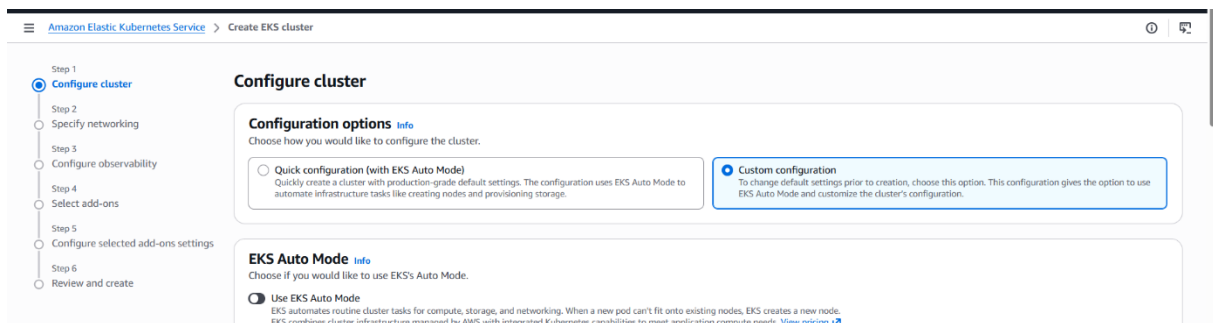
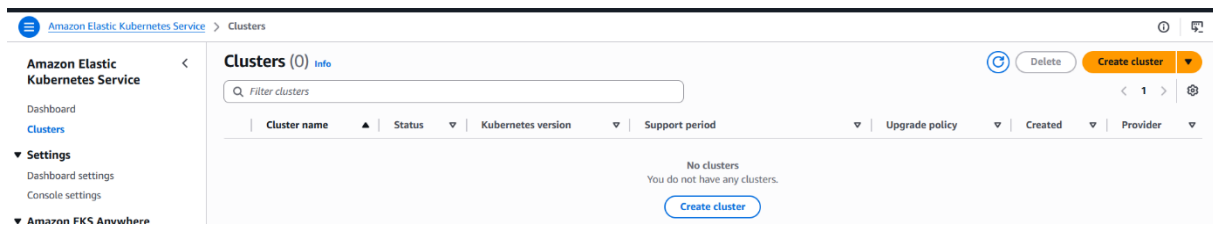
```
root@ip-172-31-11-158:/home/ubuntu#
```

CONTINUOUS DEPLOYMENT/DELIVERY PART

FINAL JENKINSFILE

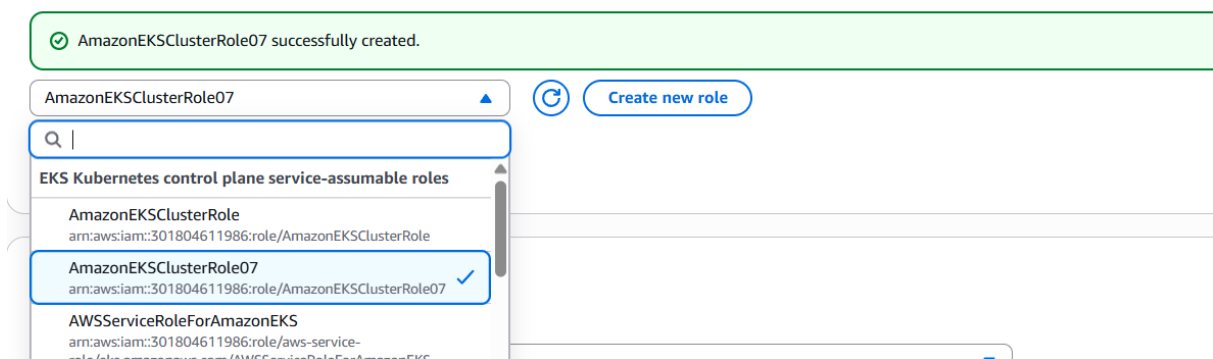
Screenshot: Create EKS Cluster

- Open Amazon EKS.
- Click **Create Cluster**.
- Select **Custom Configuration**.
- Create an IAM Role and select the role.
- Select the required VPC and Security Group.
- Set the Cluster Endpoint Access to **Public**.
- Click **Create**.
- EKS Cluster created successfully.



Cluster IAM role

To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes control plane.



Cluster authentication mode [Info](#)

Configure which source the cluster will use for authenticated IAM principals.

☐ EKS API

The cluster will source authenticated IAM principals only from EKS access entry APIs.

☒ EKS API and ConfigMap

The cluster will source authenticated IAM principals from both EKS access entry APIs and the aws-auth ConfigMap.

Additional security groups [Info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

sg-01fd0fe3f335a0753 | jai X

jai



Clear selected security groups

Choose cluster IP address family [Info](#)

Specify the IP address type for pods and services in your cluster.

☒ IPv4

☐ IPv6

Configure Kubernetes service IP address block [Info](#)

☒ Specify the range from which cluster services will receive IP addresses.

Configure remote networks to enable hybrid nodes [Info](#)

EKS Hybrid Nodes enables you to use on-premises and edge infrastructure as nodes in EKS clusters.

☒ Specify the CIDR blocks for your on-premises environments that you will use for hybrid nodes.

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

☒ Public

The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.

☐ Public and private

The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.

☐ Private

Clusters (1) [Info](#)

Filter clusters



Delete

Create cluster



< 1 > ⚙

	Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
☐	mucluster	Active	1.35 Upgrade now	Standard support until March 27, 2027	Standard support	7 minutes ago	EKS

Screenshot: Create Node Group

- Open the created EKS Cluster.
- Navigate to **Compute**.
- Click **Add Node Group**.
- Enter the name as **my-nodegrp**.
- Create and select the Node Group IAM Role.
- Select the required AMI and Instance Type.
- Configure the Node Group settings.
- Click **Create**.
- Node Group created successfully.

Name

Assign a unique name for this node group.

mynodegrp

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role

To access other AWS services and resources your Nodes needs an IAM role. Choose a role with the right permissions for your Nodes.

✓ AmazonEKSNodeRole7 successfully created.

✕

AmazonEKSNodeRole7

↕

🔄

Create new role

[Edit role](#)

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2023 (x86_64) Standard (AL2023_x86_64_STANDARD) ↕

Capacity type

Select the capacity purchase option for this node group.

On-Demand ↕

Instance types [Info](#)

Select instance types you prefer for this node group.

🔍 Enter an instance type

c7i-flex.large ✕
vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 12.5 Gigabit Max ENI: 3 Max IPs: 30

Disk size

Select the size of the attached EBS volume for each node.

20 GiB

Screenshot: Verify Worker Nodes

- Wait for the Node Group creation to complete.
- Verify that **2 Worker Nodes** are created.
- Worker Nodes created successfully.

Nodes (2) [Info](#)

🔍 Filter Nodes by property or value

< 1 >

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage
ip-172-31-46-60.ap-south-1.compute.internal	c7i-flex.large	Node group	mynodegrp	2 minutes ago	✓ Ready	12%	19%	15%
ip-172-31-5-25.ap-south-1.compute.internal	c7i-flex.large	Node group	mynodegrp	2 minutes ago	✓ Ready	32%	34%	15%

Node groups (1) [Info](#)

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

🔍 Filter node groups by property or value

< 1 >

Edit

Delete

Add

Group name	Desired size	AMI release version	Launch template	Status
mynodegrp	2	1.35.5-20260610	-	✓ Active

Screenshot: Verify EC2 Instances

- Navigate to **Instances**.
- Two Worker Node instances are running successfully.

Instances (2/3) Info Last updated less than a minute ago Connect Instance state Actions Launch instances

Saved filter sets Choose filter set

☒	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
☒		i-0c87df09cc09ec22a	Running	c7i-flex.large	3/3 checks passed	ap-south-1a	ec2-13-206-201-152.ap...	13.206.201.152	-
☐	project-server	i-02d2e529009aa0b01	Running	m7i-flex.large	3/3 checks passed	ap-south-1b	ec2-13-233-144-1.ap-s...	13.233.144.1	-
☒		i-080faa2d599ffa114	Running	c7i-flex.large	3/3 checks passed	ap-south-1b	ec2-65-1-108-35.ap-so...	65.1.108.35	-

Screenshot: Connect to EKS Cluster

- Update the kubeconfig for the EKS cluster.
- Verify the cluster connection.

```
root@ip-172-31-11-158:/home/ubuntu# aws eks update-kubeconfig --name mucluster --region ap-south-1
Added new context arn:aws:eks:ap-south-1:301804611986:cluster/mucluster to /root/.kube/config
root@ip-172-31-11-158:/home/ubuntu#
```

Screenshot: Create ArgoCD Namespace

- Create the **argocd** namespace.
- Namespace created successfully.

```
root@ip-172-31-11-158:/home/ubuntu# kubectl create namespace argocd
namespace/argocd created
root@ip-172-31-11-158:/home/ubuntu#
```

Screenshot: Create ArgoCD Values File

- Run vi values.yaml.
- Paste the ArgoCD NodePort configuration.
- Save the file.
- Values file created successfully.

```
root@ip-172-31-11-158:/home/ubuntu# vi values.yaml
root@ip-172-31-11-158:/home/ubuntu#
```

```
# Basic Argo CD values for NodePort exposure
server:
  service:
    type: NodePort
    nodePortHttp: 30080      # optional - choose a fixed NodePort (HTTP)
    nodePortHttps: 30443    # optional - choose a fixed NodePort (HTTPS)
    ports:
      http: 80
      https: 443
  ingress:
    enabled: false
  insecure: true             # serve UI without enforcing HTTPS (for lab use)
  extraArgs:
    - --insecure             # disable TLS redirection in ArgoCD server

configs:
  cm:
    # Disable TLS redirect so UI works over plain HTTP
    server.insecure: "true"

redis:
  enabled: true

controller:
  replicas: 1
repoServer:
  replicas: 1
applicationSet:
  replicas: 1
```

Screenshot: Add Argo Helm Repository

- Add the Argo Helm repository.
- Update the Helm repositories.
- Argo Helm repository configured successfully.

```
root@ip-172-31-11-158:/home/ubuntu#  
root@ip-172-31-11-158:/home/ubuntu# helm repo add argo https://argoproj.github.io/argo-helm  
"argo" has been added to your repositories  
root@ip-172-31-11-158:/home/ubuntu# helm repo update  
Hang tight while we grab the latest from your chart repositories...  
...Successfully got an update from the "argo" chart repository  
Update Complete. ★Happy Helming!★
```

Screenshot: Install ArgoCD

- Install ArgoCD using the custom values file.
- ArgoCD installed successfully.

```
Update Complete. ★Happy Helming!★  
root@ip-172-31-11-158:/home/ubuntu# helm upgrade --install argocd argo/argo-cd \\  
-n argocd \\  
-f values.yaml  
Release "argocd" does not exist. Installing it now.  
NAME: argocd  
LAST DEPLOYED: Tue Jun 16 06:58:55 2026  
NAMESPACE: argocd  
STATUS: deployed
```

Screenshot: Verify ArgoCD Pods

- Check the pods in the **argocd** namespace.
- Verify all pods are in the **Running** state.

```
root@ip-172-31-11-158:/home/ubuntu#  
root@ip-172-31-11-158:/home/ubuntu# kubectl get pods -n argocd  
NAME                                READY   STATUS    RESTARTS   AGE  
argocd-application-controller-0     1/1     Running   0           3m29s  
argocd-applicationset-controller-6959ff54fc-zd878  1/1     Running   0           3m30s  
argocd-dex-server-6f74798d97-ns2gd  1/1     Running   0           3m30s  
argocd-notifications-controller-5f57f69879-dz6rz  1/1     Running   0           3m30s  
argocd-redis-7858d89dd5-v46sn       1/1     Running   0           3m30s  
argocd-repo-server-59d7744c5-pdhtp   1/1     Running   0           3m30s  
argocd-server-7f7d7f4995-r6l82      1/1     Running   0           3m30s  
root@ip-172-31-11-158:/home/ubuntu#
```

Screenshot: Get Worker Node Public IP

- Open the EC2 Dashboard.
- Copy the Public IP of any Worker Node.

Instance summary for **i-0c87df09cc09ec22a** [Info](#)

Updated less than a minute ago

Instance ID

 i-0c87df09cc09ec22a

IPv6 address

—

Public IPv4 address

 13.206.201.152 | [open address](#)

Instance state

 Running

Private IPv4 address

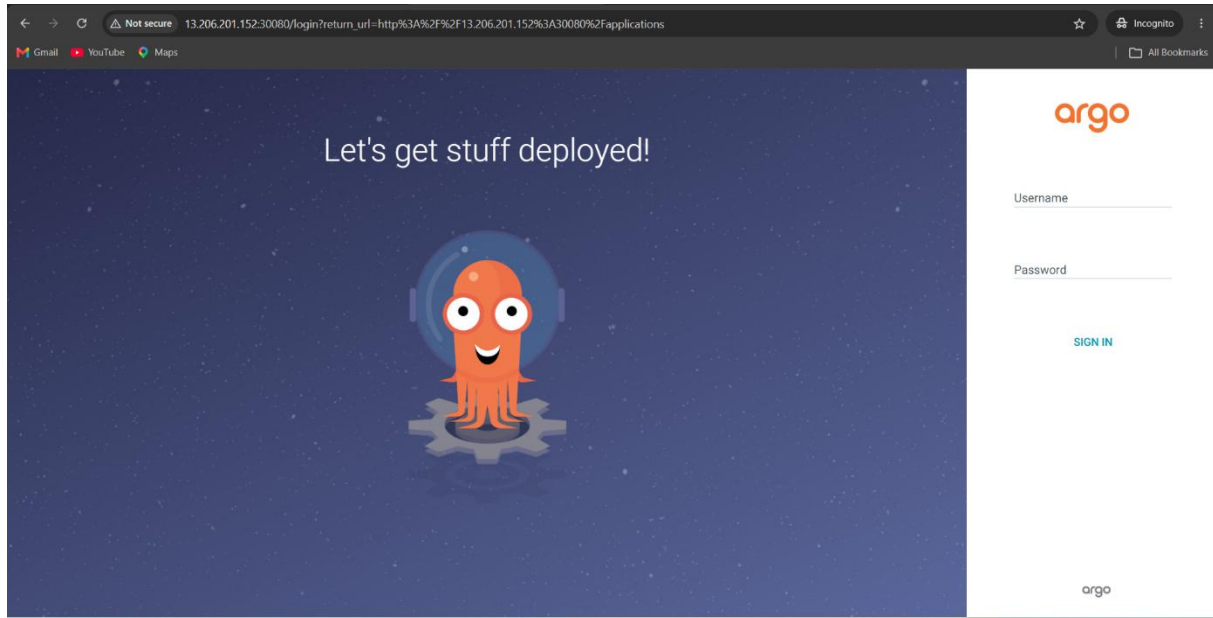
 —

Public IPv6 address

 —

Screenshot: Access ArgoCD UI

- Open a new browser tab.
- Navigate to **Public-IP:30080**.
- ArgoCD Sign-In page displayed successfully.



Screenshot: Get ArgoCD Initial Password

- Retrieve the **argocd-initial-admin-secret**.
- Copy the encoded password.

```
root@ip-172-31-11-158:/home/ubuntu# kubectl -n argocd get secret argocd-initial-admin-secret -o yaml
apiVersion: v1
data:
  password: S3k1N3RHZXdlTnZGLUZQbg==
kind: Secret
metadata:
  creationTimestamp: "2026-06-16T06:59:16Z"
  name: argocd-initial-admin-secret
  namespace: argocd
  resourceVersion: "3302"
```

Screenshot: Decode ArgoCD Password

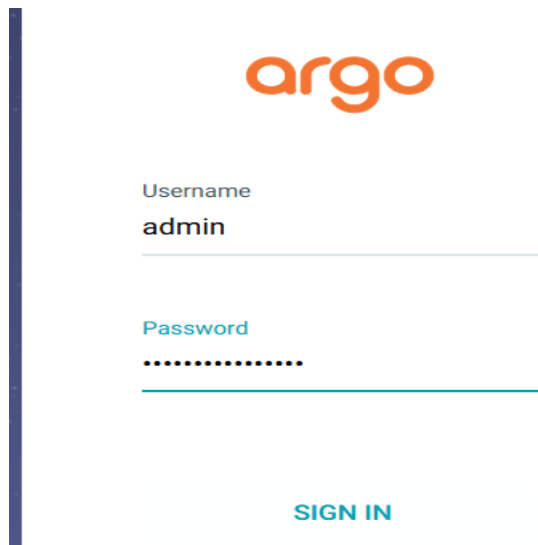
- Decode the Base64 encoded password.
- Copy the decoded password.

```
type: Opaque
root@ip-172-31-11-158:/home/ubuntu# echo S3k1N3RHZXdlTnZGLUZQbg== | base64 --decode
```

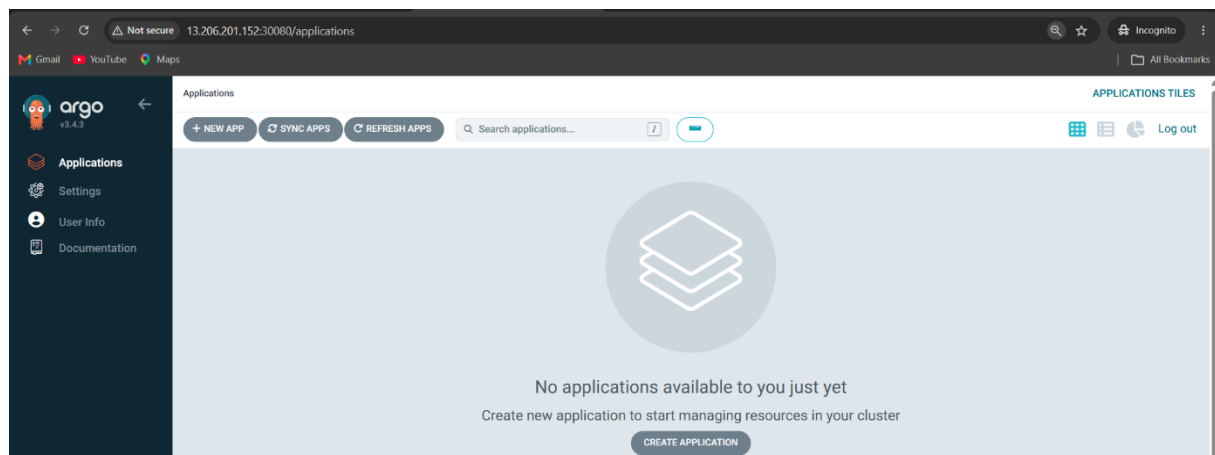
```
root@ip-172-31-11-158:/home/ubuntu# echo S3k1N3RHZXdlTnZGLUZQbg== | base64 --decode
Ky57tGeweNvF-FPnroot@ip-172-31-11-158:/home/ubuntu#
```

Screenshot: Login to ArgoCD

- Enter the username as **admin**.
- Paste the decoded password.
- Click **Sign In**.
- ArgoCD Dashboard opened successfully.

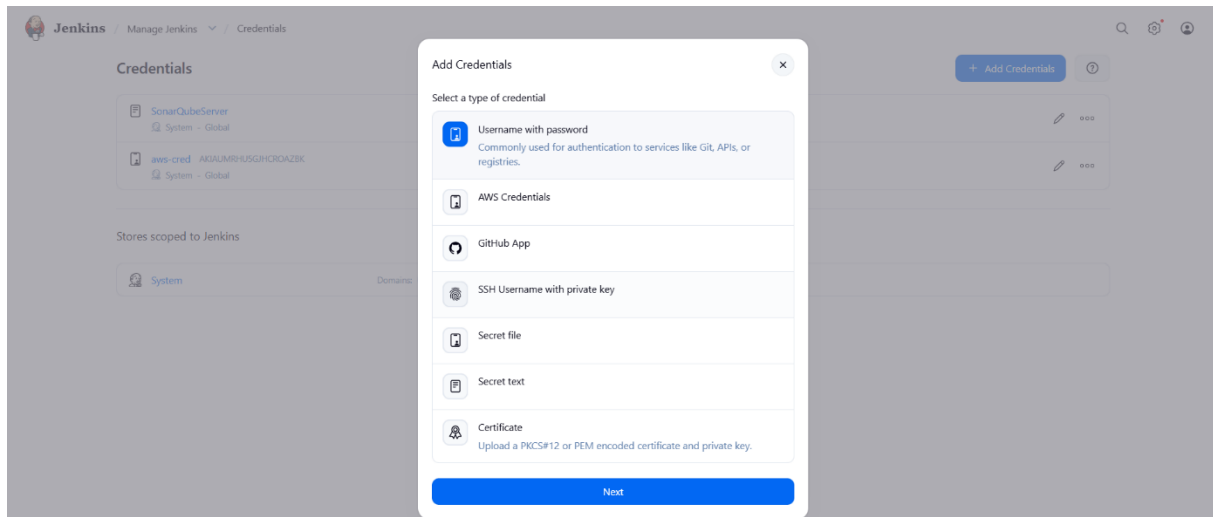


The screenshot shows the ArgoCD login interface. At the top is the orange 'argo' logo. Below it, the text 'Username' is followed by an input field containing 'admin'. Underneath, the text 'Password' is followed by an input field filled with dots. At the bottom, there is a blue 'SIGN IN' button.



Screenshot: Add GitHub Credentials in Jenkins

- Open **Manage Jenkins**.
- Click **Credentials**.
- Click **Add Credentials**.
- Select **Username with Password**.



Screenshot: Create GitHub Credential

- Enter the GitHub username.
- Enter the GitHub password or personal access token.
- Enter the ID as **git-cred**.(same as in jenkins script)
- Click **Create**.
- GitHub credential created successfully.

A screenshot of the 'Add Username with password' form in Jenkins. The form has a title bar with a back arrow, the title 'Add Username with password', and a close button (X). Below the title bar, there is a 'Scope' dropdown menu set to 'Global (Jenkins, nodes, items, all child items, etc)'. The 'Username' field contains the text 'jaisurya003'. There is a checkbox labeled 'Treat username as secret' which is currently unchecked. The 'Password' field is a text input with masked characters (dots). The 'ID' field contains the text 'git-cred'. There is a 'Description' field which is empty. At the bottom of the form is a blue 'Create' button.

Screenshot: Verify GitHub Credential

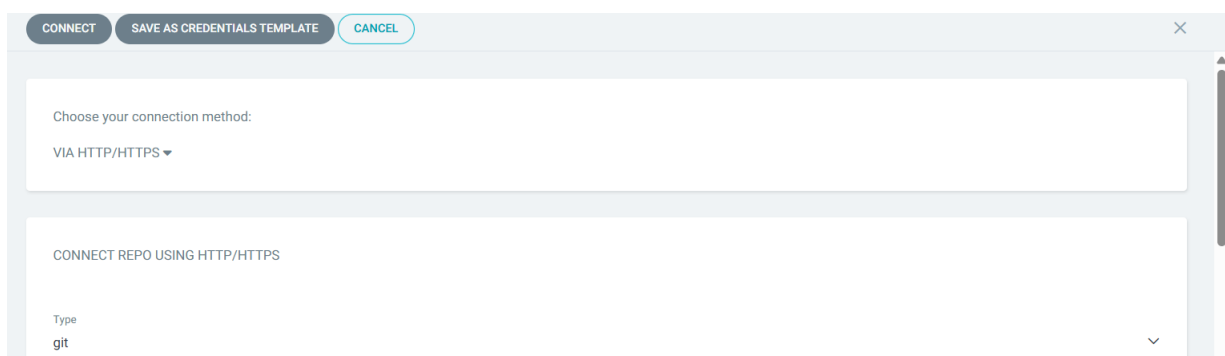
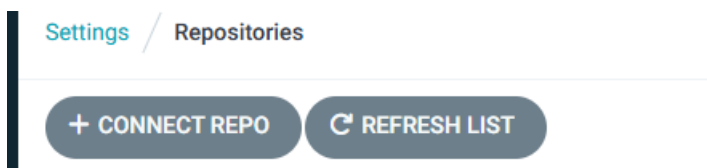
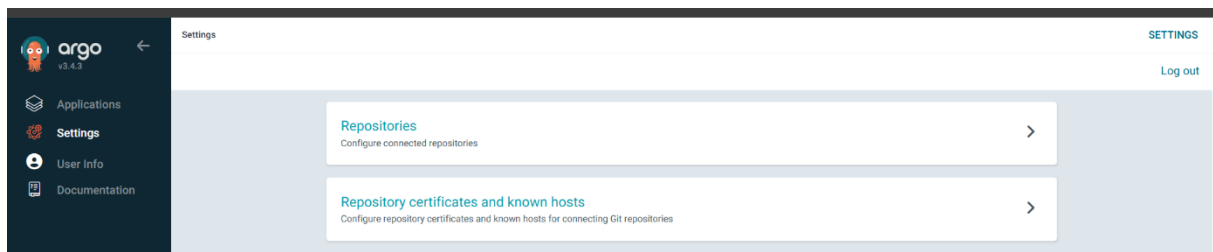
- Open **Manage Jenkins** → **Credentials**.
- Verify that **git-cred** is available.

Credentials

	SonarQubeServer System - Global
	aws-cred AKIAUMRHU5GJHCROAZBK System - Global
	git-cred jaisurya003/***** System - Global

Screenshot: Connect Repository in ArgoCD

- Open the **ArgoCD Dashboard**.
- Navigate to **Settings** → **Repositories**.
- Click **Connect Repo**.
- Select **Via HTTPS**.
- Select **Git** as the repository type.
- Enter the project name as **default**.



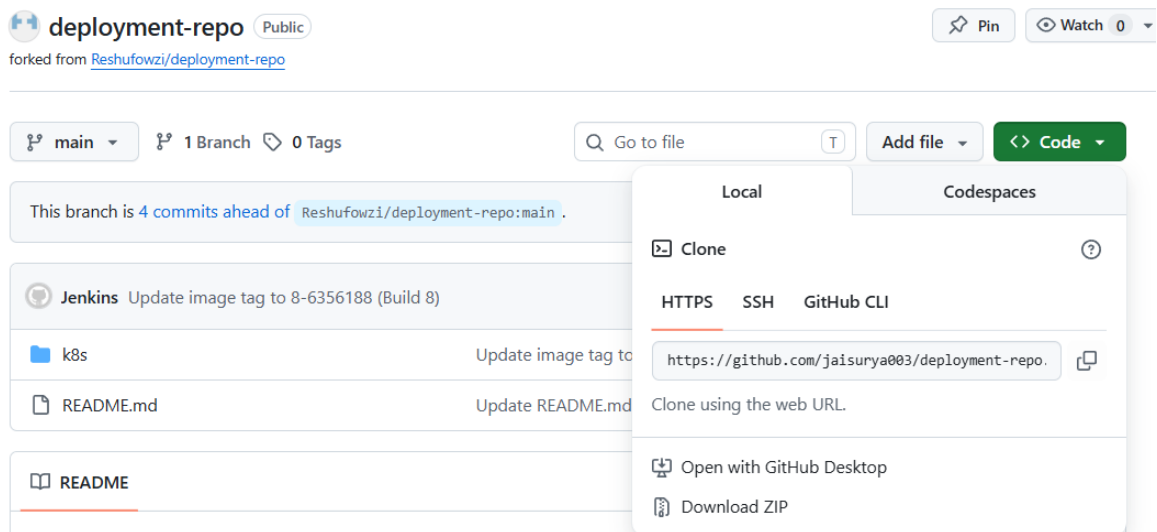
Screenshot: Configure Repository Details

- Enter the Git repository URL containing the Kubernetes deployment and service manifests.
- Click **Connect**.

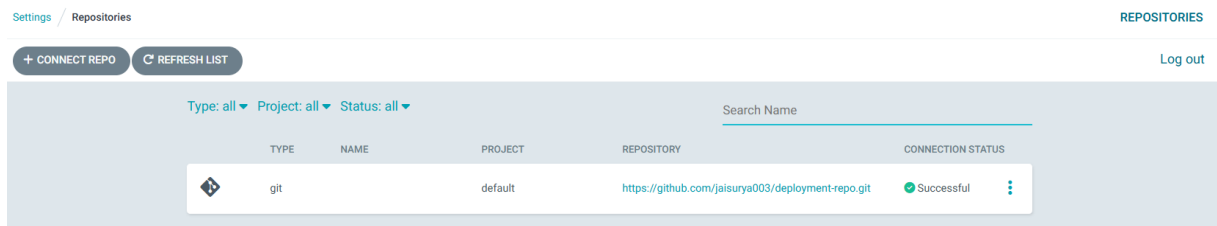
Project
default

Repository URL

<https://github.com/jaisurya003/deployment-repo.git>



Repository connected successfully.



Screenshot: Create ArgoCD Application

- Enter the application name as **my-project**.
- Select the project as **default**.
- Enable **Automatic Sync Policy** by selecting all required checkboxes.
- Continue to the application configuration page.

GENERAL

Application Name
my-project

Project Name
default

SYNC POLICY
Automatic

☒ ENABLE AUTO-SYNC

☒ PRUNE RESOURCES

☒ SELF HEAL

EDIT AS YAML

Screenshot: Configure Application Path

- Enter the repository path as **k8s** (folder containing Deployment and Service manifests).

SOURCE

Repository URL
https://github.com/jaisurya003/deployment-repo.git

Revision
HEAD

Path
k8s

GIT ✓

Branches ▼

deployment-repo / k8s

Jenkins Update image tag to 8-6356188 (Build 8) bdacd66 · 8 minutes ago History

This branch is 4 commits ahead of ReshuFowzi/deployment-repo:main

Contribute Sync fork

Name	Last commit message	Last commit date
..		
deployment.yaml	Update image tag to 8-6356188 (Build 8)	8 minutes ago
service.yaml	Update service.yaml	3 months ago

- Select the target cluster.
- Enter the namespace as **default**.
- Click **Create**.
- ArgoCD application created successfully.

DESTINATION

Cluster URL

https://kubernetes.default.svc

URL ▼

Namespace

default

Screenshot: Copy ECR Repository URI

- Open **Amazon ECR**.
- Select the repository.
- Open the latest image.
- Copy the Repository URI.
- ECR Repository URI copied successfully.

Image

Details

Image tags

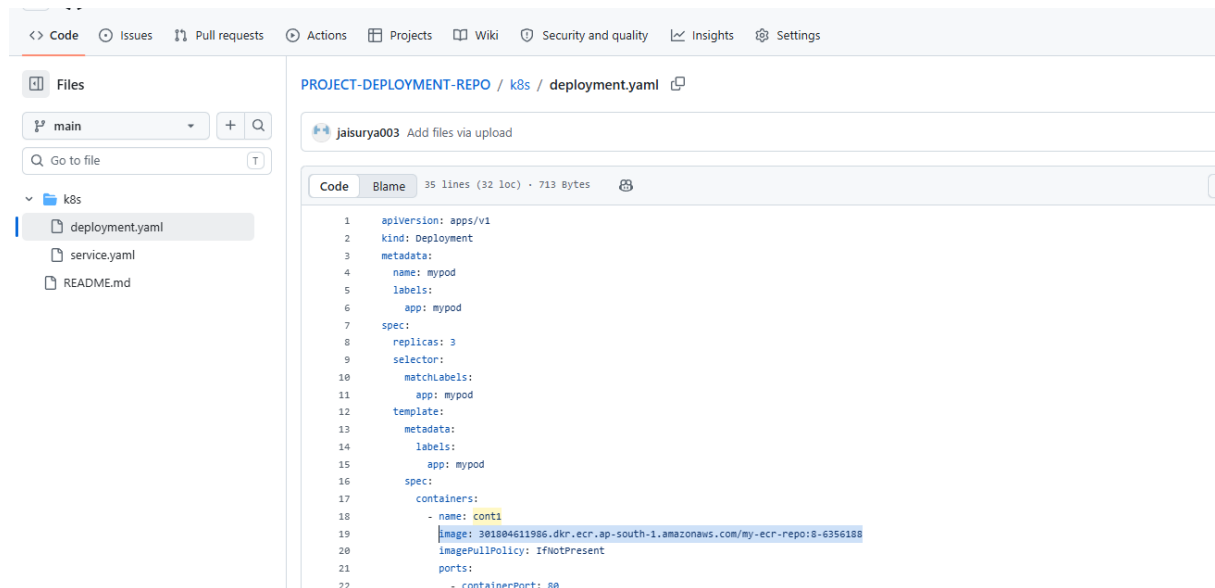
Repository URI copied



301804611986.dkr.ecr.ap-south-1.amazonaws.com/my-ecr-repo:5-9947c21

Screenshot: Update Deployment Manifest

- Open the Deployment Repository.
- Edit the **deployment.yaml** file.
- Paste the ECR Image URI in the image field.
- Save the changes.
- Deployment manifest updated successfully.



Screenshot: Update Jenkins Pipeline

- Open **my-project** in Jenkins.
- Click **Configure**.
- Replace the existing pipeline with the final Jenkins pipeline containing all stages.
- Click **Apply**.
- Click **Save**.
- Jenkins pipeline updated successfully.

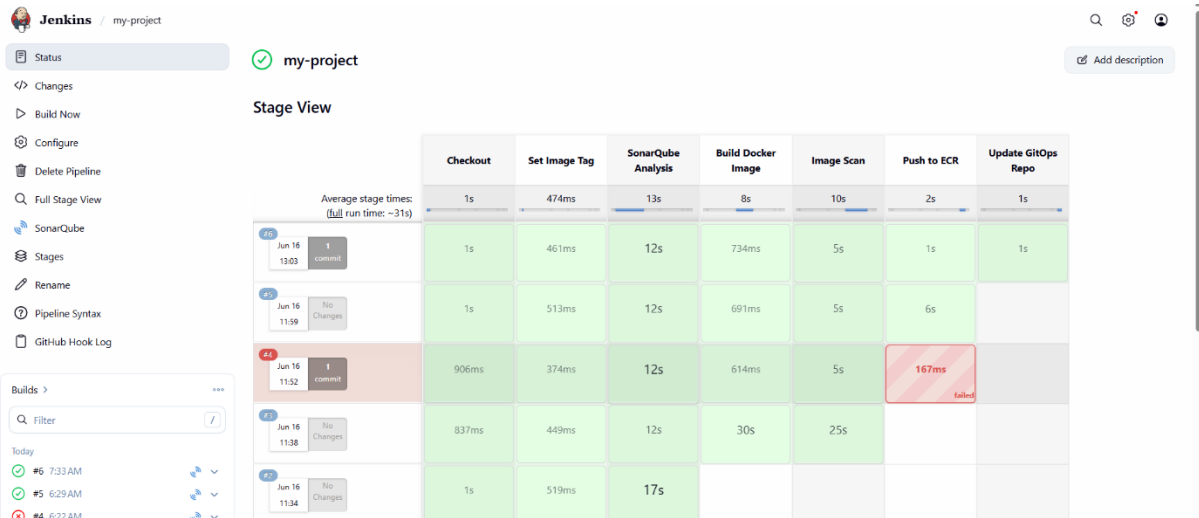
```

1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = 'mydockerimage'
6     SONARQUBE_SERVER = 'SonarQubeServer'
7     PATH = "/opt/sonar-scanner/bin:${PATH}"
8     ECR_REPO = '301804611986.dkr.ecr.ap-south-1.amazonaws.com/my-ecr-repo'
9     AWS_REGION = 'ap-south-1'
10    DEPLOYMENT_REPO = 'https://github.com/jaisurya003/deployment-repo.git'
11  }
12
13  stages {
14
15    stage("Checkout") {

```

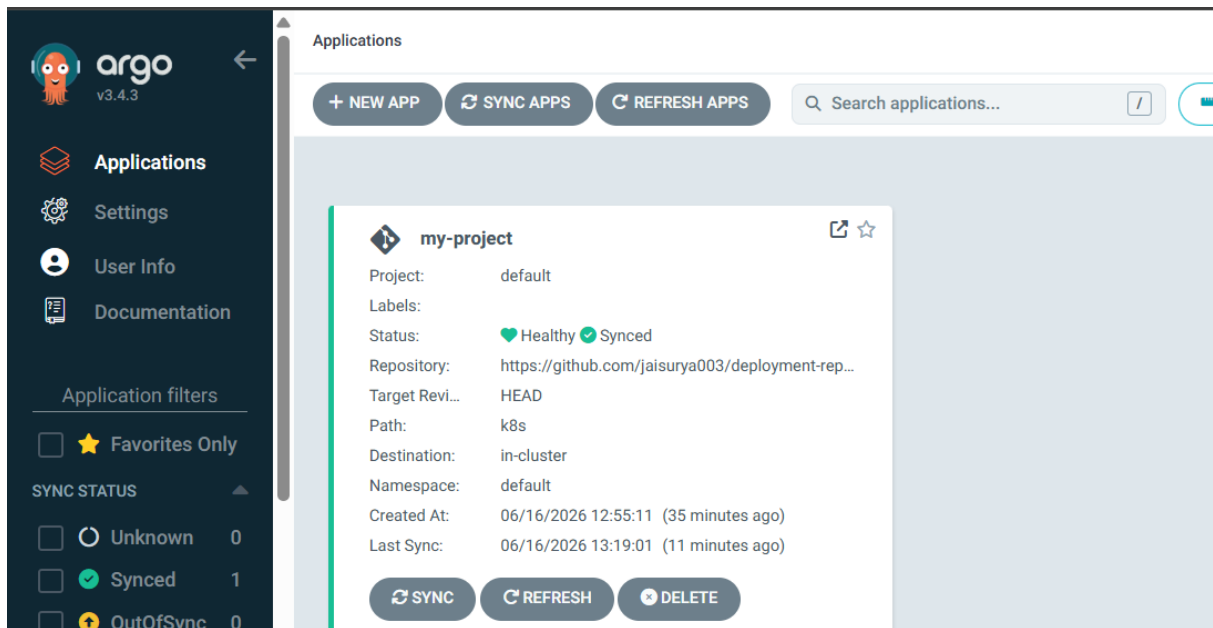
Screenshot: Run Complete CI/CD Pipeline

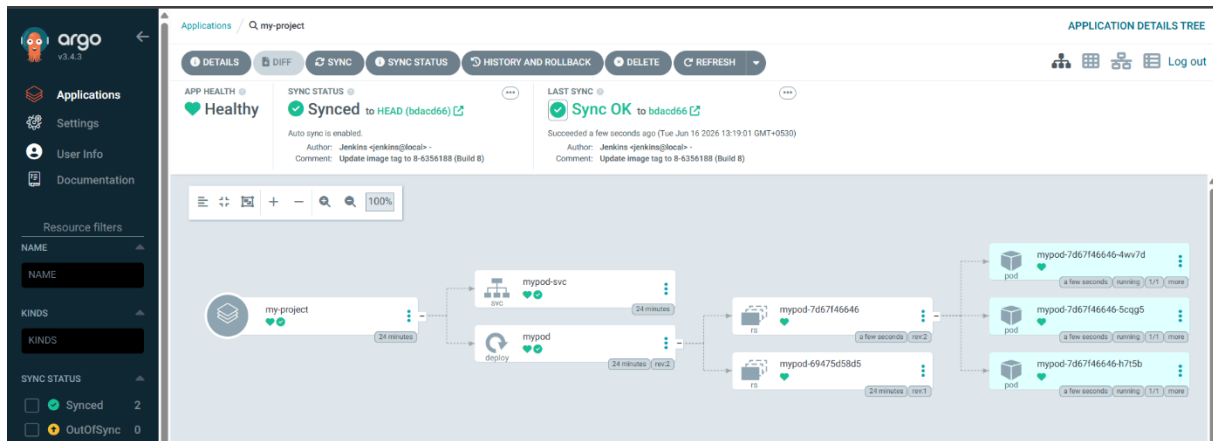
- Open **my-project**.
- Click **Build Now**.
- Verify that all stages completed successfully, including **Update GitOps Repo**.



Screenshot: Verify ArgoCD Synchronization

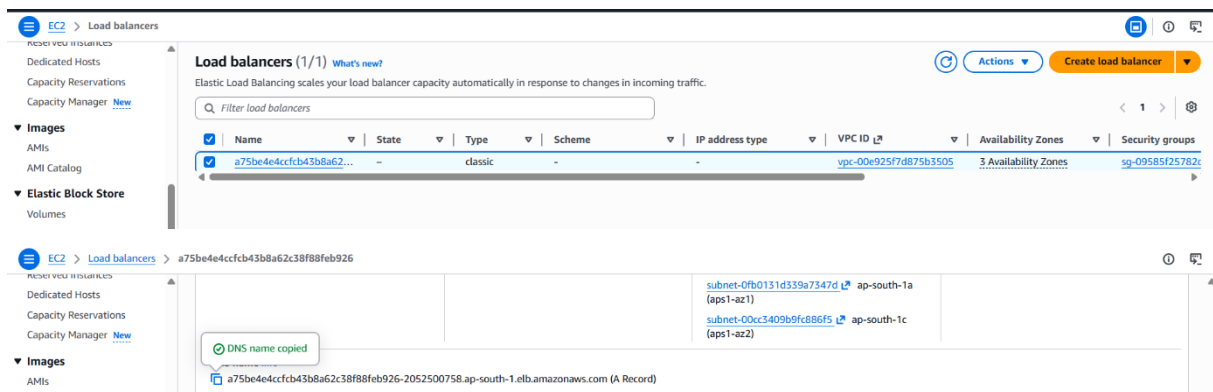
- Open the **ArgoCD Dashboard**.
- Open **my-project**.
- Verify that all resources are **Synced** and **Healthy**.
- GitOps deployment completed successfully.





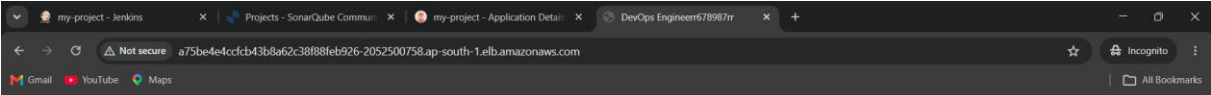
Screenshot: Verify AWS Load Balancer

- Open the AWS Console.
- Navigate to **Load Balancers**.
- Verify that the Load Balancer is created automatically.
- Copy the DNS Name of the Load Balancer.
- Load Balancer created successfully.



Screenshot: Access Application

- Open a new browser tab.
- Paste the Load Balancer DNS Name.
- Application content displayed successfully.
- End-to-end CI/CD deployment completed successfully.



Resma G (DevSecOps Engineer) reshma

SLA Institute

Building Scalable, Automated & Reliable Infrastructure

About Me

DevOps Engineer currently working with **SLA Institute**. I specialize in building CI/CD pipelines, cloud infrastructure, and automation solutions that help organizations deliver applications faster, reliably, and efficiently.

Technical Skills

Category	Technologies
Cloud Platforms	AWS, Microsoft Azure
Operating Systems	Linux
Version Control	Git, GitHub, GitLab CI
CI/CD Tools	Jenkins, Maven
Configuration Management	Ansible
Containerization	Docker, Kubernetes
Monitoring Tools	Prometheus, Grafana